

PhiGraph Core 4.0: A Shadow-First Evidence Ledger and Policy-Gated Runtime for Software-Agent Operations

Walter Calmels von Dem Knesebeck
TUCH Systems, Santiago, Chile
wcalmels@phi47.cl

July 2026

Abstract

Software agents and artificial-intelligence systems can produce claims, recommendations, and action proposals faster than organizations can verify and govern them. This paper presents PhiGraph Core 4.0.0, an MIT-licensed, model-agnostic system that treats model output as a proposal rather than as verified truth. PhiGraph represents claims, evidence, verifications, action proposals, policy decisions, and outcomes as linked records; stores them in a scope-tagged, tamper-evident ledger; and places policy evaluation between agent proposals and operational execution. Replay and shadow modes cannot execute external actions, while higher-authority modes require both an explicit executor and an allowing policy decision. We describe the protocol, runtime, ledger, deployment boundaries, and Python implementation. Engineering verification comprises 117 automated tests; the continuous-integration configuration targets Python 3.10–3.13. An initial external experiment on the CIC-IDS2017 Friday DDoS flow file compares a PhiGraph relational score with three unsupervised baselines. Local Outlier Factor is strongest on this tabular benchmark; PhiGraph ranks second by PR-AUC and attains 0.840 precision at a 10% alert budget. The experiment demonstrates functional integration, not state-of-the-art superiority: it uses a single imperfect dataset, a feature-similarity graph, and one fixed split. PhiGraph is therefore presented as an auditable governance substrate and research prototype, not as a proof of correctness, causal identification, or safe autonomous operation.

Keywords: AI governance; software agents; provenance; evidence ledger; graph intelligence; policy enforcement; shadow deployment; reproducible software

1 Introduction

Modern software agents can synthesize text, modify code, recommend operational changes, invoke tools, and coordinate multi-step workflows. Their outputs may be useful without being reliable, authorized, or supported by reproducible evidence. This creates a control problem: an organization must determine what was claimed, which evidence supported or refuted it, which policy governed a proposed action, who approved it, and what happened after evaluation or execution.

Existing guidance emphasizes lifecycle risk management and traceability. The NIST AI Risk Management Framework organizes activities around governing, mapping, measuring, and managing AI risk [9]. W3C PROV provides a general vocabulary for provenance [4], while model cards and dataset datasheets improve documentation of released artifacts [6, 2]. These approaches are important, but they do not by themselves define an operational boundary between a model-generated proposal and an authorized action.

PhiGraph addresses this boundary through an evidence-oriented protocol and a policy-gated runtime. Its central invariant is that model output is a candidate claim or action proposal, not verified truth or implicit authority. The system records relationships among claims, evidence, verification results, policies, approvals, and outcomes so that a decision can be reconstructed after the fact.

This paper makes four contributions:

1. a typed protocol for claims, evidence, verifications, action proposals, policy decisions, and outcomes;
2. a runtime in which policy evaluation and an explicit executor are necessary conditions for external action;
3. a backend-neutral, scope-tagged ledger with hash-chain integrity metadata and optional HMAC evidence signatures; and
4. a bounded empirical evaluation comprising software verification and an external anomaly-ranking experiment, together with explicit threats to validity.

2 Related Work

2.1 Governance, documentation, and provenance

The NIST AI RMF is a voluntary, use-case-agnostic framework for managing AI risk [9]. PhiGraph is not an implementation or certification of that framework; it provides operational records that may support governance activities. PROV-O formalizes entities, activities, agents, and provenance relations [4]. PhiGraph uses a narrower application protocol centered on verification and policy-gated action. A future interoperability layer could map PhiGraph records to PROV-O.

Model cards [6] and datasheets for datasets [2] make intended uses, evaluation conditions, and limitations explicit. PhiGraph applies a related transparency principle at runtime: each claim and action remains linked to its evidence, decision, and outcome rather than relying only on static release documentation.

2.2 Relational representations and software assurance

Graph-based representations make entities and relationships first-class and support structured reasoning [1]. PhiGraph uses heterogeneous graphs and classical graph analytics; it does not require a graph neural network and does not claim that graph structure alone establishes causality. In software assurance, in-toto protects software-supply-chain integrity through verifiable step metadata [10]. PhiGraph shares an emphasis on attributable records but targets a broader claim–evidence–policy–action lifecycle.

Agentic systems introduce risks including tool misuse, privilege compromise, untraceable actions, and cascading errors. The OWASP Agentic Security Initiative provides an industry threat taxonomy [7]. PhiGraph’s deny-by-default policy, approval requirements, idempotency controls, and shadow-first deployment are engineering controls relevant to these risks; they are not a complete security solution.

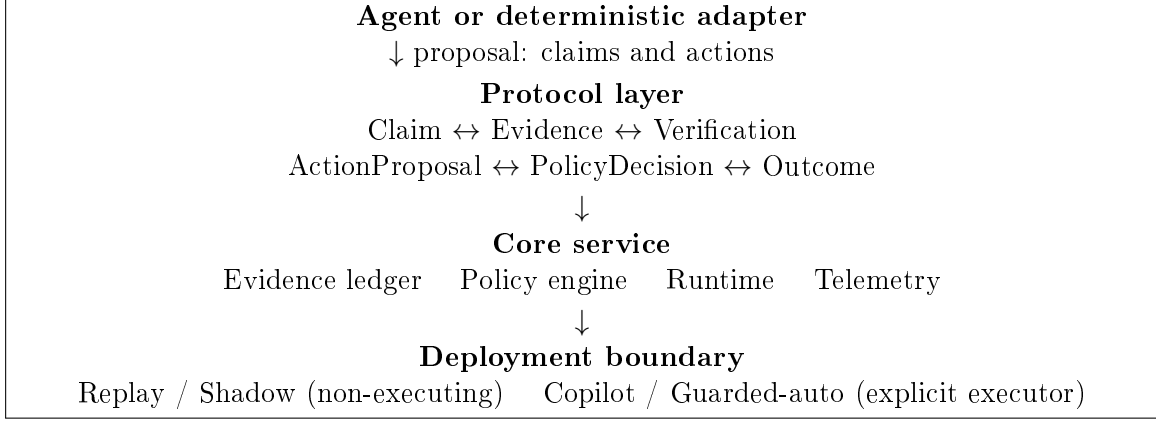


Figure 1: PhiGraph separates agent proposals from verification, policy evaluation, and execution. Arrows indicate recorded processing relationships, not automatic trust propagation.

3 System Model and Design Goals

Let an agent produce a proposal

$$P = (C, A),$$

where C is a set of claims and A is a set of proposed actions. A claim does not become verified merely because an agent assigns it high confidence. Instead, a verifier records a result over a claim and an explicit evidence set. Similarly, an action is not authorized by its presence in P : the policy engine evaluates the action under a runtime mode and an approval set.

PhiGraph is designed around six goals:

1. **Evidence boundedness:** when evidence identifiers are supplied, verification results reference evidence already registered in the ledger.
2. **Least authority:** absence of a matching policy blocks an action.
3. **Traceability:** claims, decisions, and outcomes remain reconstructable.
4. **Safe evaluation:** replay and shadow runs never execute an external action.
5. **Scope tagging:** tenant and project identifiers propagate through stored records and can be used to filter queries.
6. **Model neutrality:** deterministic programs, statistical models, and language-model agents can use the same protocol.

4 PhiGraph Architecture

4.1 Canonical protocol

Protocol version 2.0.0 exposes six principal frozen dataclass record schemas:

- **Claim:** statement, type, subject, issuer, status, confidence, evidence references, and optional supersession;
- **Evidence:** kind, source, payload, status, content hash, observation time, and metadata;

- **Verification**: claim, verifier, method, result, evidence references, rationale, and time;
- **ActionProposal**: action type, target, proposer, parameters, rationale claims, reversibility, and risk level;
- **PolicyDecision**: effect, applicable policies, reasons, required approvals, and time; and
- **Outcome**: action, status, whether execution occurred, observed effects, evidence references, and time.

Claim states distinguish proposed, unverified, partially verified, verified, refuted, and superseded records. Policy effects distinguish allow, warn, require approval, and block. These states make uncertainty and governance decisions explicit instead of collapsing them into a single confidence score. The current implementation does not require a non-empty evidence set for every verified claim and does not assess evidentiary sufficiency. This is an enforcement gap, not a guarantee supplied by the protocol.

4.2 Evidence ledger

The ledger supports JSON, SQLite, and PostgreSQL backends through a common interface. Collections are maintained for claims, evidence, verifications, actions, policy decisions, and outcomes. Most records are appended. Claim verification performs a controlled in-place state transition and then rebuilds the affected collection chains. Each ordinary append computes

$$h_i = \text{SHA256}(h_{i-1} \parallel k \parallel \text{canonical}(r_i)),$$

where r_i is a record and k identifies its collection. Verification checks link continuity and recomputes each hash. Evidence payloads receive a SHA-256 content hash and may be signed with HMAC-SHA256 when a signing key is supplied.

Chain verification detects changes relative to the currently stored chain. This mechanism is *tamper evident*, not tamper proof: without an external anchor or protected signing key, it cannot distinguish malicious tampering from an authorized rebuild. An attacker with write access to both the records and all integrity keys can rebuild a consistent chain. Stronger deployment requires external key management, append-only storage, access control, backups, and independent audit anchors.

4.3 Policy-gated runtime

The policy engine matches action type, permitted runtime modes, maximum risk, reversibility, and required approvals. No matching rule produces an implicit deny decision. For an action a , decision d , mode m , and executor x , the core execution condition is

$$\text{execute}(a) \iff (d.\text{effect} = \text{allow}) \wedge (m \in \{\text{copilot}, \text{guarded_auto}\}) \wedge (x \neq \emptyset).$$

Replay and shadow modes therefore generate simulated outcomes with `executed=false`. The packaged deployment configuration is stricter: it defaults to shadow-only operation, rejects attempts to disable that boundary, and disables real connectors. The core abstractions expose higher-authority modes for controlled future integration, but the supplied deployment does not grant arbitrary external execution.

4.4 API, SDK, and operations

The Python SDK can send tenant and project headers, API-key or bearer credentials, and idempotency keys. Authentication, PostgreSQL persistence, and signing are supported by lower-level Core components when explicitly configured. The packaged application does not currently wire every one of these options into the Core v3 router. Its Docker configuration must therefore be treated as a local evaluation setup unless protected by an external authentication and network boundary. Scope headers are application metadata and query filters; the present implementation does not provide end-to-end tenant isolation or same-scope referential-integrity guarantees.

5 Evaluation

5.1 Research questions

The evaluation addresses two bounded questions:

- RQ1** Is the software version mechanically testable across its supported Python versions and packaging paths?
- RQ2** Does the current relational anomaly score produce useful rankings on one external network-flow benchmark relative to simple unsupervised baselines?

It does not test whether PhiGraph prevents all unsafe agent actions, proves causality, or outperforms specialized state-of-the-art intrusion-detection systems.

5.2 Software verification

The software tree contains 117 automated tests covering protocol records, ledger integrity, policy evaluation, runtime modes, persistence backends, API and SDK boundaries, agent workflows, graph operations, benchmarks, and deployment configuration. Continuous integration is configured to install the declared extras and run the suite on Python 3.10, 3.11, 3.12, and 3.13; separate jobs build the Python distribution, install its wheel in an isolated environment, and build the Docker image. A local run on July 27, 2026 executed all 117 tests successfully. A separate synthetic benchmark attempt under local Python 3.14, which is outside the CI matrix, stopped with an ARPACK eigensolver error and is excluded from the reported empirical results. Passing tests establish conformance to encoded cases, not correctness for all deployments.

5.3 External anomaly-ranking experiment

The committed external-validation artifact uses the CIC-IDS2017 Friday afternoon DDoS flow file [8]. It contains 225,745 rows: 97,718 benign and 128,027 DDoS. With random seed 47, the protocol draws a 12,000-row benign-only reference set and evaluates a balanced test set of 6,000 benign and 6,000 DDoS flows. Preprocessing and variable-feature selection use only the benign reference. Methods are ranked continuously and evaluated at a top-10% alert budget.

The compared methods are robust multivariate deviation, Isolation Forest, Local Outlier Factor (LOF), and a PhiGraph relational score combining robust deviation, mean nearest-neighbor distance, and disagreement between their percentile ranks. Table 1 reproduces the committed result.

Table 1: CIC-IDS2017 Friday DDoS anomaly-ranking results. Runtime is reported by the committed validation artifact; its execution environment was not recorded.

Method	ROC-AUC	PR-AUC	P@10%	R@10%	F1@10%	Time (s)
Local Outlier Factor	0.9757	0.9506	0.9617	0.1923	0.3206	0.322
PhiGraph relational	0.7295	0.7316	0.8400	0.1680	0.2800	0.165
Isolation Forest	0.7403	0.6840	0.7817	0.1563	0.2606	0.321
Robust deviation	0.5771	0.5540	0.4117	0.0823	0.1372	0.006

LOF is clearly strongest on this experiment. PhiGraph is second by PR-AUC and precision at budget, but its ROC-AUC is slightly below Isolation Forest. This single fixed experiment establishes only that the implementation produces a non-degenerate ranking under the reported protocol. It does not support a claim of general or state-of-the-art superiority.

6 Threats to Validity and Limitations

Dataset validity. CIC-IDS2017 has documented problems in flow construction, feature generation, and labels [5, 3]. Results may change on corrected data. The machine-learning CSV also omits source and destination IP addresses, timestamps, users, devices, and processes. The evaluated graph is therefore a feature-similarity graph rather than the heterogeneous operational graph for which PhiGraph is designed.

Experimental validity. The external result uses one file, one attack family, one seed, and one alert budget. It reports no uncertainty intervals, ablation confidence intervals, or prospective evaluation. The benchmark is balanced and does not reproduce a production base rate. The PhiGraph score is batch-transductive because its component percentile ranks are calculated over the complete evaluation batch; scores can therefore change with batch composition and are not calibrated online anomaly scores. Additional chronological splits, repeated seeds, corrected datasets, heterogeneous event sources, and cost-sensitive metrics are required.

Software validity. Automated tests can encode incorrect assumptions and do not substitute for penetration testing, formal verification, independent replication, or operational incident analysis. The ledger’s hash chain detects accidental or unauthorized changes only under an appropriate trust and key-management model. JSON and SQLite are evaluation backends; production use requires hardened storage, secrets management, authorization, monitoring, and backup procedures.

Causal and governance claims. Graph connectivity, spectral structure, and anomaly scores do not identify causal effects without assumptions and suitable interventions. PhiGraph records candidate causal pathways but does not prove causality from observational data. Likewise, a recorded approval may still be mistaken or malicious. The platform improves inspectability and enforcement points; it does not guarantee ethical, legal, or correct outcomes.

7 Reproducibility and Availability

PhiGraph Core 4.0.0 is implemented in Python and distributed under the MIT License. Its development repository is maintained at <https://github.com/wcalmels/phigraph>. A versioned public

archive should be created together with the Zenodo record. The paper source and references are licensed under CC BY 4.0.

The software test suite can be reproduced with:

```
python -m venv .venv
pip install -e ".[api,benchmark,dev,auth,app]"
pytest
```

The CIC-IDS2017 experiment requires the Friday DDoS CSV and is run with:

```
python scripts/validate_cicids2017.py /path/to/Friday...DDos.csv
```

The committed validation artifact records metrics and predictions but does not record the source-dataset checksum, exact dependency versions, hardware, operating system, or Git revision. It is therefore an auditable result artifact, not yet an exact reproducibility package. A final archive should preserve the dataset provenance and checksum, Python and dependency versions, seed, configuration, hardware description, and Git revision. The Zenodo DOI should be added to this section and to the repository citation metadata after the archival deposit is published.

8 Ethical and Operational Considerations

PhiGraph can store sensitive evidence, identity metadata, and operational proposals. Deployers must minimize collection, define retention periods, encrypt transport and storage, separate tenant access, protect signing keys, and audit privileged operations. Security telemetry and network datasets may contain personal or confidential information and require an appropriate legal basis. The system should be evaluated in replay or shadow mode before any higher-authority integration. Human approval should remain mandatory where consequences are difficult to reverse or affect safety, rights, employment, finance, health, or critical infrastructure.

9 Conclusion

PhiGraph makes a specific architectural choice: intelligence does not imply authority. Claims must remain linked to evidence and verification; actions must remain subject to policy and explicit execution boundaries; outcomes must remain auditable. Version 4.0.0 demonstrates this design through a typed protocol, tamper-evident ledger, policy-gated runtime, scope-tagged records, and an automated test suite. The external CIC-IDS2017 experiment shows a functional relational anomaly score but also illustrates why bounded interpretation is necessary: a conventional LOF baseline performs best, and the dataset cannot exercise PhiGraph’s full heterogeneous graph model. Future work should prioritize independent replication, corrected and chronological security datasets, multi-source operational graphs, policy-adversarial testing, cryptographically anchored audit storage, and prospective shadow deployments.

Author Contributions

Walter Calmels von Dem Knesebeck conceived the system, implemented the software, designed the evaluation, analyzed the results, and wrote the manuscript.

Conflict of Interest

The author is affiliated with TUCH Systems, the organization developing PhiGraph. This relationship may create a commercial interest in the system. The evaluation and limitations are reported to enable independent scrutiny.

License

This paper is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

References

- [1] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- [2] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [3] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In *Risks and Security of Internet and Systems*, pages 18–33, 2022.
- [4] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. W3C recommendation, World Wide Web Consortium, 2013.
- [5] Lisa Liu, Gints Engelen, Timothy Lynar, Daryl Essam, and Wouter Joosen. Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018. In *2022 IEEE Conference on Communications and Network Security*, pages 254–262, 2022.
- [6] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.
- [7] OWASP Agentic Security Initiative. Agentic ai: Threats and mitigations. Technical report, OWASP Foundation, 2025.
- [8] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pages 108–116, 2018.
- [9] Elham Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.
- [10] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410. USENIX Association, 2019.